

UNIVERSIDAD DE BURGOS

Curso: 2026-2027

VALIDACIÓN Y PRUEBAS

PROYECTO TEORICO 1

Descripción breve

Informe de la resolución del proyecto teórico 1.

Rubén Castañeda Matute y Ariadna Ortega Fernández

INDICE

1.INTRODUCCIÓN 3
1.1.Objetivos 3
1.2.Metodología seguida 4
2.DESCRIPCIÓN DE LA APLICACIÓN 5
2.1.Funcionalidades principales 5
2.2.Usuarios del sistema 5
2.3.Alcance de las pruebas 5
3.PROYECTO CREADO EN TESTLINK 6
3.1.Estructura definida 6
3.2.Justificación de las decisiones adoptadas 13
4.INVENTARIO DE ENTIDADES (PARA CADA ENTIDAD) 14
4.1.Entidad 1 14
4.2.Entidad 2 15
4.3.Entidad 3 16
4.4.Entidad 4 17
4.5.Entidad 5 18
4.6.Entidad 6 19
4.7.Entidad 7 20
4.8.Entidad 8 21
4.9.Entidad 9 22
4.10.Entidad 10 23
5.DIAGRAMA ER 24
5.1.Diagrama completo 24
5.2.Cardinalidades 25
5.3.Hipótesis realizadas 26
5.3.1.Pertenencia de suites y casos de prueba 26
5.3.2.Versionado de los casos de prueba 26
5.3.3.Relación entre planes y casos de prueba 26
5.3.4.Relación entre proyectos, planes y builds 27
5.3.5.Registro de múltiples ejecuciones 27
5.3.6.Resultados individuales de los pasos 27
5.3.7.Participación de los usuarios 27
5.3.8.Cardinalidades mínimas 28
5.4.Relaciones observadas 29

6.REFLEXIÓN TEMPORAL 30
6.1.Valoración crítica 30
6.2.Propuestas de mejora 30
6.3.Lecciones aprendidas 31

1.INTRODUCCIÓN

1.1.Objetivos

El objetivo principal de este trabajo es comprender los conceptos fundamentales de la gestión de pruebas y la forma en que estos se relacionan dentro de una herramienta profesional, desarrollando una visión global del ciclo de vida de las pruebas de software. Para lograr esto, utilizaremos la máquina virtual proporcionada por el profesor con el software TESTLink como herramienta de apoyo para la creación y gestión de un proyecto de pruebas.

De forma específica, nuestro grupo aplicará estos conceptos sobre la "Aplicación 6. Gestión de eventos universitarios", un sistema diseñado para permitir crear eventos, registrar participantes y controlar aforos.

A partir de la construcción de un proyecto sencillo basado en esta aplicación, los objetivos secundarios incluyen:

- ➔ Generar la información mínima necesaria para documentar las pruebas.
- ➔ Analizar el funcionamiento de la herramienta TESTLink e identificar las entidades que intervienen en su gestión, así como sus propiedades y relaciones.
- ➔ Elaborar un modelo conceptual que represente la estructura interna de la herramienta mediante un diagrama Entidad-Relación (ER).

1.2.Metodología seguida

Para la realización de este trabajo y la consecución de los objetivos, se seguirá una metodología basada en la experimentación y observación directa del comportamiento de TESTLink, dado que no está permitido consultar su esquema de base de datos ni su documentación técnica interna. Las fases metodológicas son las siguientes:

- ***Comprensión del sistema:*** Analizaremos la aplicación de gestión de eventos universitarios para identificar sus funcionalidades principales, los tipos de usuarios que interactuarán con ella y determinaremos el alcance de las pruebas.
- ***Construcción del proyecto en TESTLink:*** Utilizando la máquina virtual proporcionada, crearemos la estructura mínima requerida: 1 proyecto de pruebas, 1 versión, 3 suites de pruebas y entre 10 y 12 casos de prueba.
- ***Planificación y ejecución:*** Definiremos 1 plan de pruebas, asociaremos los casos diseñados y ejecutaremos las pruebas registrando los resultados correspondientes, asegurándonos de obtener casos superados, fallidos y bloqueados o no ejecutados. Durante este proceso, prestaremos especial atención a la información que solicita la herramienta.
- ***Identificación de entidades y modelado conceptual:*** A partir de la observación durante el uso de TESTLink, identificaremos las entidades principales, sus atributos relevantes y las relaciones entre ellas.
- ***Elaboración del diagrama ER:*** Finalmente, construiremos el diagrama Entidad-Relación, justificando las decisiones adoptadas y las cardinalidades mediante ejemplos extraídos directamente de nuestra interacción con la herramienta durante las pruebas de la aplicación de eventos.

2.DESCRIPCIÓN DE LA APLICACIÓN

La aplicación 6, Gestión de eventos universitarios, es un sistema que permite a los organizadores crear eventos, modificar sus datos, eliminarlos y consultar los eventos existentes. Los participantes pueden inscribirse en los eventos disponibles y consultar el estado de sus inscripciones. El sistema controla el aforo máximo de cada evento, impidiendo que se registren más participantes de los permitidos y validando que el aforo sea mayor que cero.

2.1.Funcionalidades principales

- ***Gestión de eventos:*** creación de eventos con datos válidos, modificación de eventos existentes, eliminación de eventos y consulta de eventos registrados.
- ***Registro de participantes:*** inscripción de participantes en eventos, validación de datos obligatorios, prevención de inscripciones duplicadas y control del número de plazas disponibles.
- ***Control de aforo:*** gestión del número máximo de participantes, control de plazas disponibles, impedimento de inscripción cuando no quedan plazas y validación de que el aforo de un evento no sea igual a cero.

2.2.Usuarios del sistema

- ***Administrador/organizador:*** usuario encargado de crear, modificar y eliminar eventos, así como de configurar el aforo máximo y supervisar las inscripciones.
- ***Participante/estudiante:*** usuario que consulta los eventos disponibles, se inscribe en ellos y consulta el estado de sus inscripciones.

2.3.Alcance de las pruebas

El alcance de las pruebas se centra en validar las funcionalidades principales de la aplicación mediante 12 casos de prueba organizados en 3 suites: Gestión de Eventos (4 casos), Registro de Participantes (4 casos) y Control de Aforo (4 casos). Se probará la creación de eventos con datos válidos y la validación de campos obligatorios, la modificación y eliminación de eventos, la inscripción de participantes, la prevención de duplicados, el control de plazas disponibles y la restricción de crear eventos con aforo igual a cero. Quedan fuera del alcance las pruebas de rendimiento, seguridad, accesibilidad, usabilidad avanzada e integraciones con otros sistemas.

3.PROYECTO CREADO EN TESTLINK

Para documentar el proyecto de pruebas de la aplicación Gestión de eventos universitarios se ha utilizado TESTLink como herramienta de gestión. A continuación, se describe la estructura creada y se justifican las decisiones adoptadas.

3.1.Estructura definida

La estructura jerárquica creada en TESTLink es la siguiente:

- ***Proyecto de pruebas:*** ***GEU:*** Gestión de eventos universitarios.
- ***Versión:*** ***GEU v1.0.***
- ***Suites de pruebas:*** 3 suites, organizadas según las funcionalidades principales de la aplicación:
  - Gestión de Eventos (4 casos de prueba).
  - Registro de Participantes (4 casos de prueba).
  - Control de Aforo (4 casos de prueba).
- ***Casos de prueba:*** 12 casos en total, distribuidos en las tres suites anteriores. Los identificadores y títulos son los siguientes:
  - ***GEU-1:*** Crear Evento con Datos Válidos.
  - ***GEU-2:*** Crear Evento Sin Datos Obligatorios.
  - ***GEU-3:*** Modificar un Evento Existente.
  - ***GEU-4:*** Eliminar un Evento Existente.
  - ***GEU-5:*** Registrar Participante con Datos Válidos.
  - ***GEU-6***: Inscribir Participante con Datos Obligatorios Incompletos.
  - ***GEU-7***: Evitar Inscripción Duplicada de Participante.
  - ***GEU-8:*** Consultar Participantes Inscritos en un Evento.
  - ***GEU-9:*** Inscribir Participante con una Única Plaza Disponible.
  - ***GEU-10:*** Impedir Inscripciones con Aforo Completo.
  - ***GEU-11:*** Comprobar Número de Plazas Disponibles.
  - ***GEU-12:*** Impedir Crear Evento con Aforo Igual a Cero.
- ***Plan de pruebas:*** Plan de Pruebas: Gestión de Eventos Universitarios.
- ***Build:*** ***GEU v1.0.***

➔ ***Ejecuciones:*** se han ejecutado los 12 casos de prueba, registrando resultados de tipo Passed (superado), Failed (fallido) y Blocked (bloqueado), tal como se observa en las capturas incluidas.

Las capturas de pantalla que acompañan a este apartado muestran la creación del proyecto, la versión, las suites, los casos de prueba, el plan de pruebas, el build y las ejecuciones con sus respectivos estados.

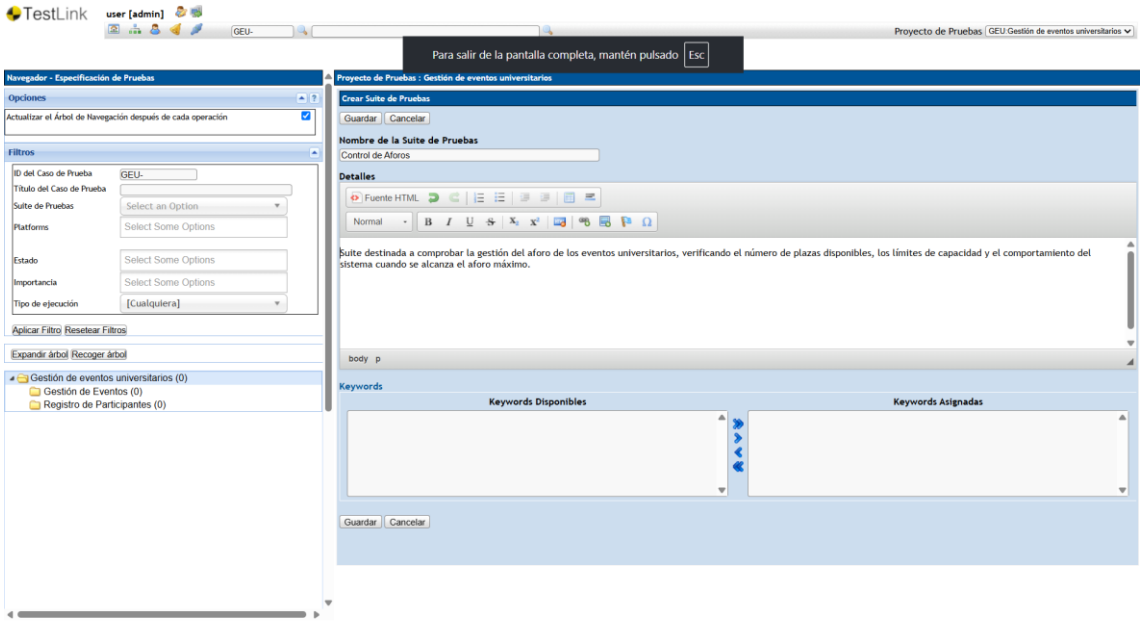


Ilustración 1 - Creación de una suite de pruebas

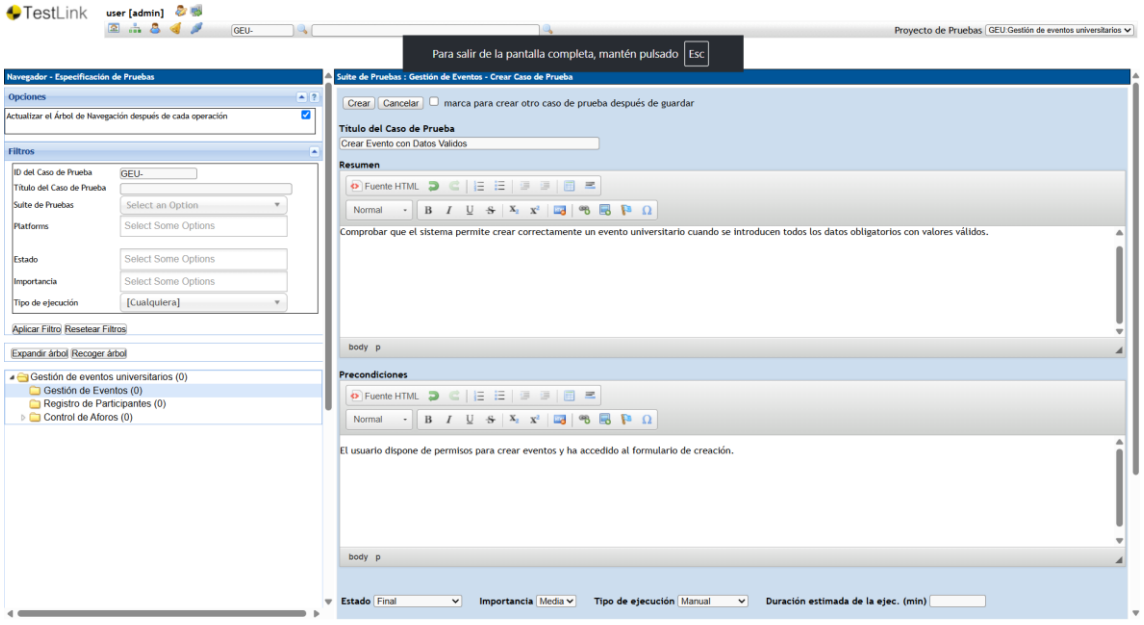


Ilustración 2 - Creación de un caso de prueba

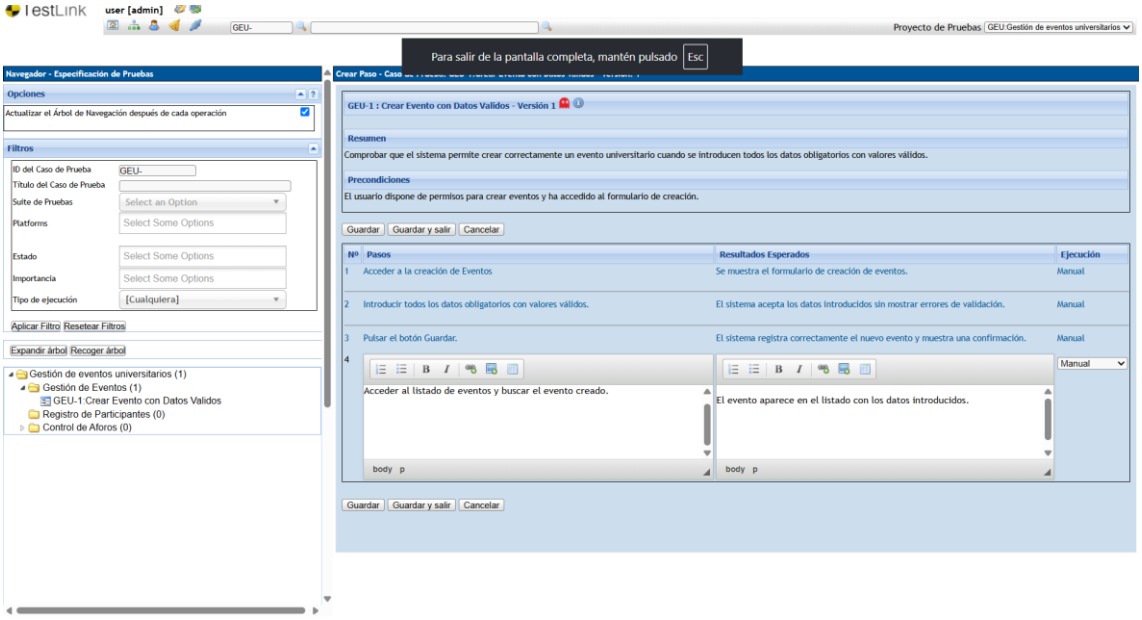


Ilustración 3 - Creación de un caso de prueba 2

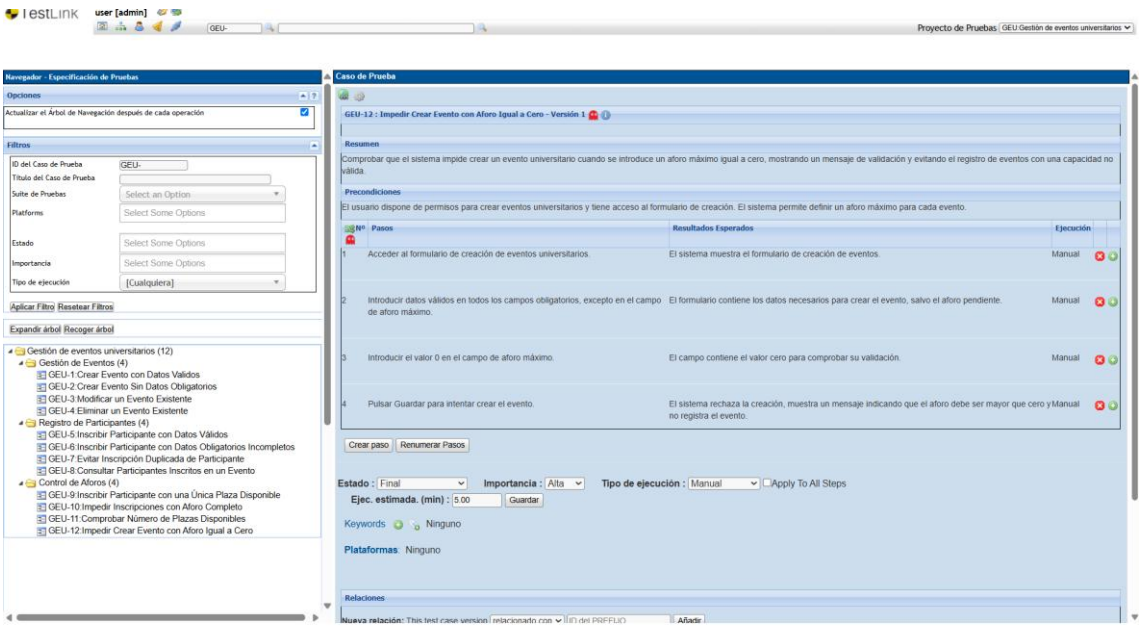


Ilustración 4 - Creación de un caso de prueba 3

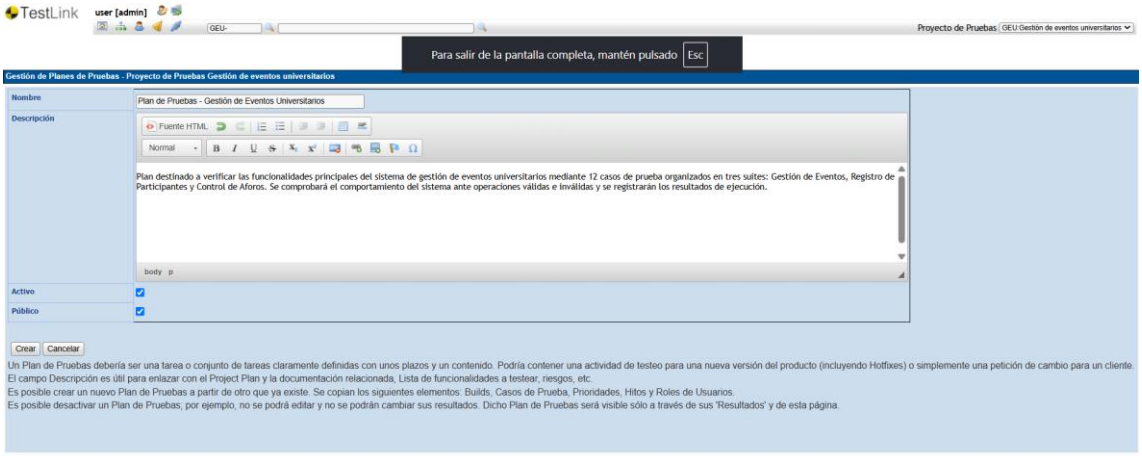


Ilustración 5 - Plan de pruebas 1

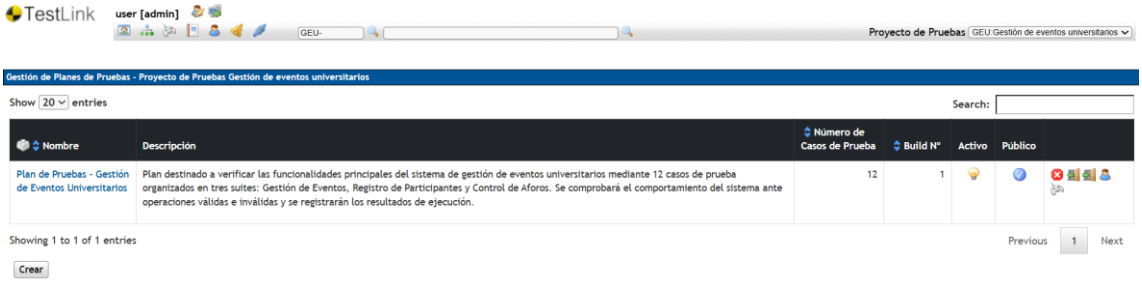


Ilustración 6 - Plan de pruebas 2

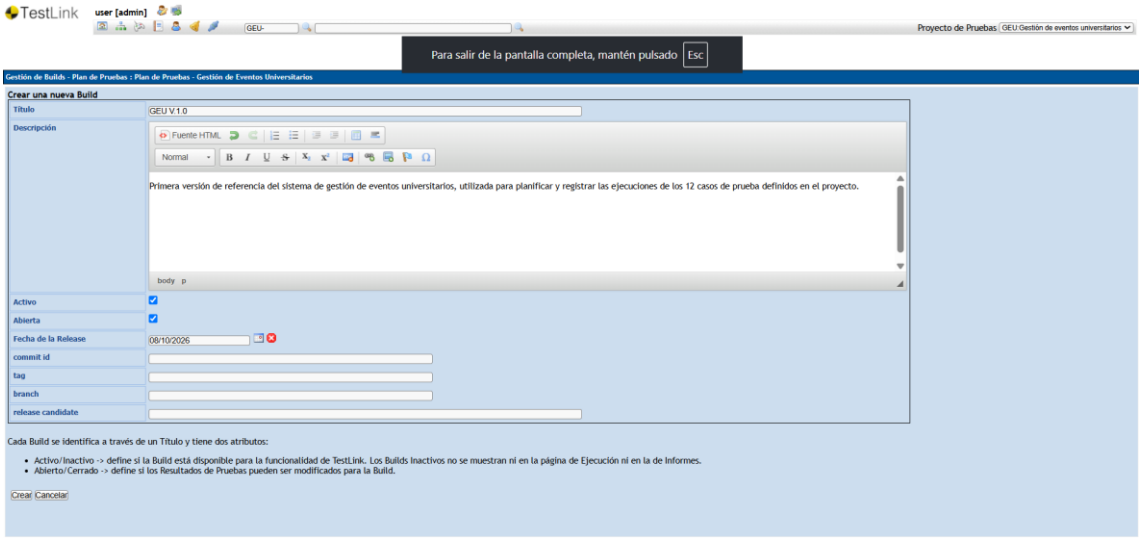


Ilustración 7 - Build GEU v1.0.

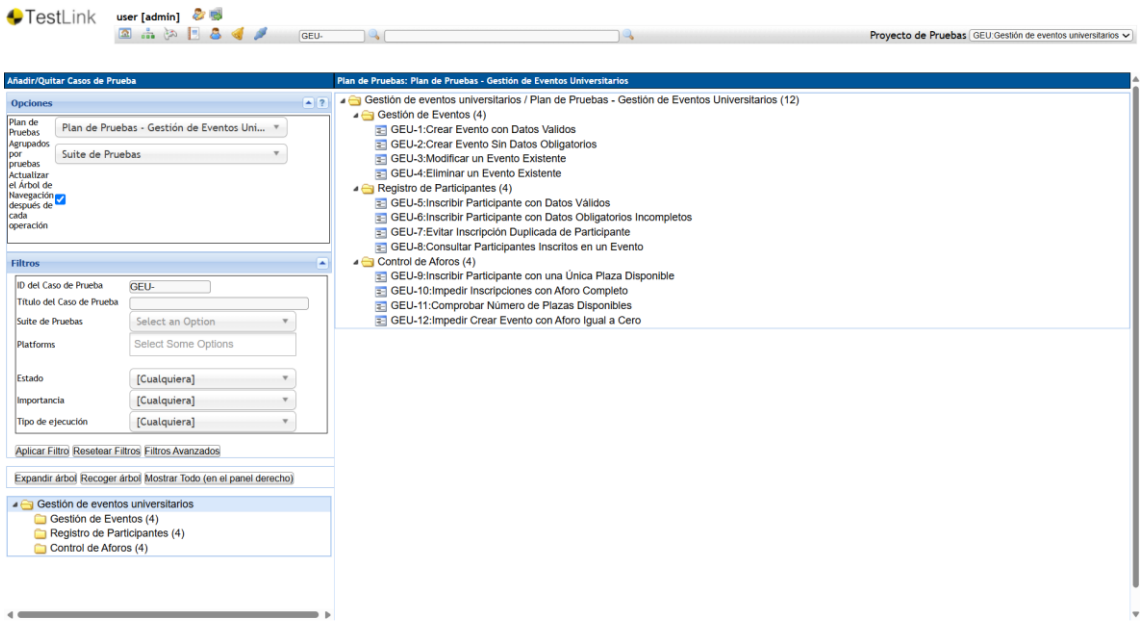


Ilustración 8 - Resumen de casos

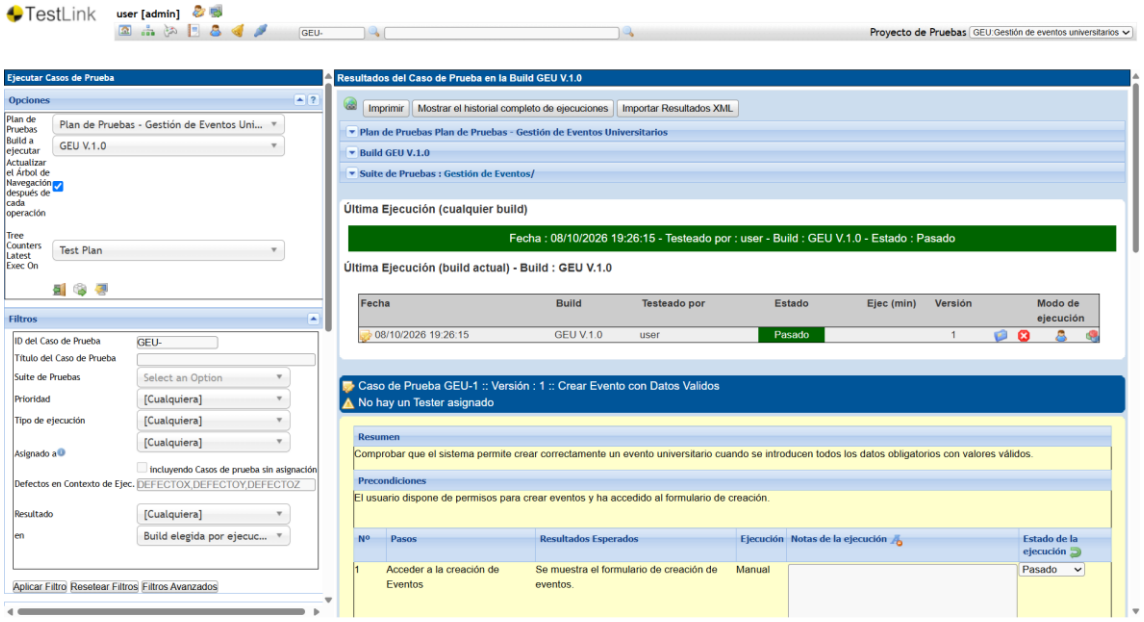


Ilustración 9 - Ejecución superada (Passed)

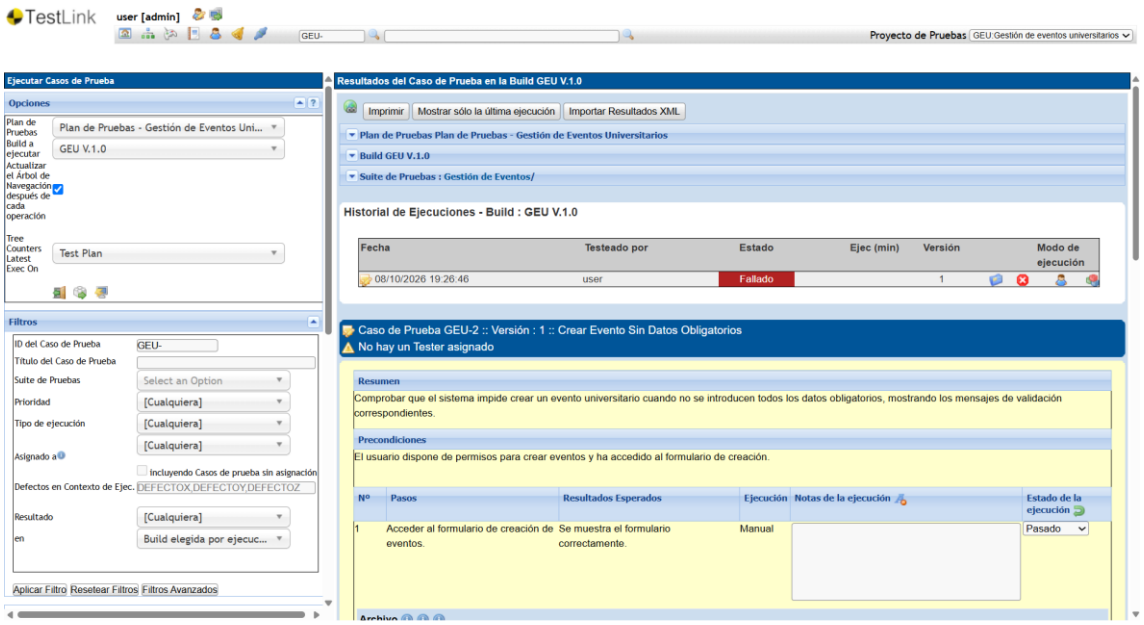


Ilustración 10 - Ejecución fallida (Failed)

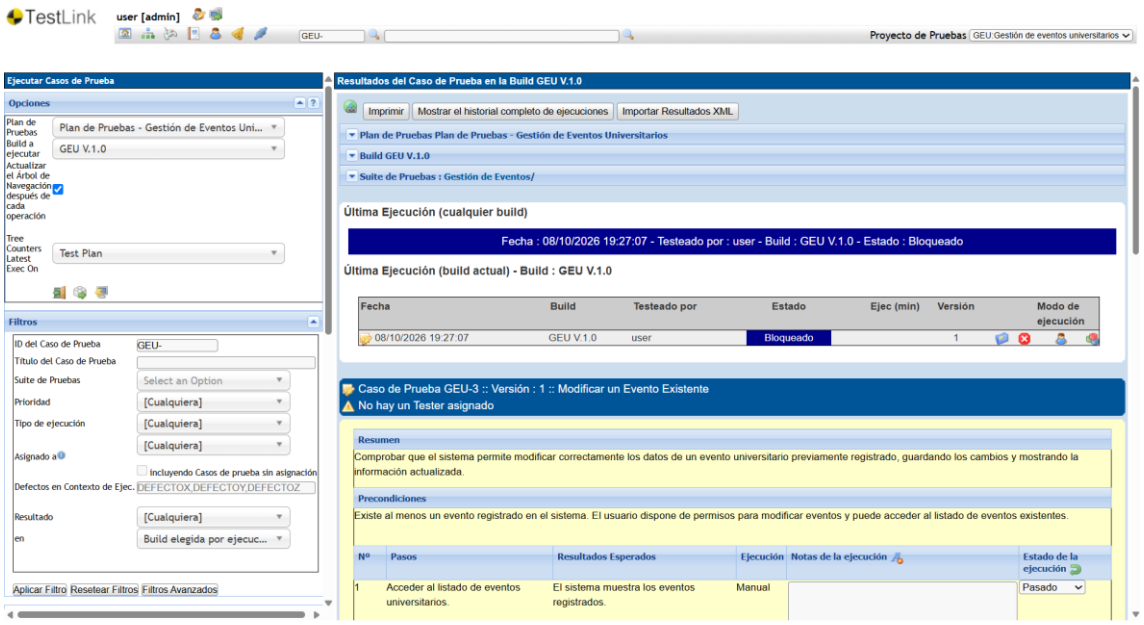


Ilustración 11 - Ejecución bloqueada (Blocked)

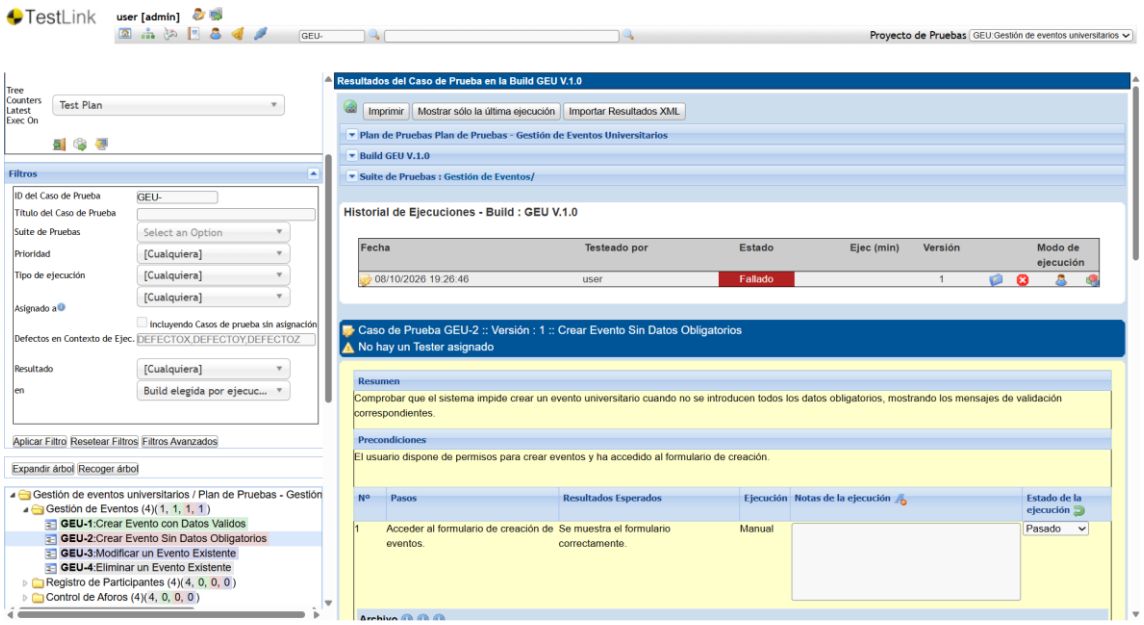


Ilustración 12 - Ejecución de los tres estados en el árbol

3.2.Justificación de las decisiones adoptadas

La estructura creada responde a los requisitos mínimos del enunciado y a la naturaleza de la aplicación probada. Las decisiones adoptadas se justifican a continuación:

- ***Un único proyecto de pruebas:*** se creó un solo proyecto (GEU: Gestión de eventos universitarios) porque el trabajo se centra en una única aplicación. Esto permite mantener centralizada toda la información y evitar duplicidades.
- ***Una única versión:*** se definió la versión ***GEU v1.0*** para representar el estado inicial de la aplicación sobre el que se ejecutan las pruebas. El enunciado exige al menos una versión, y una sola es suficiente para este proyecto teórico.
- ***Tres suites de pruebas:*** se decidió organizar los casos en tres suites coincidiendo con las tres funcionalidades principales de la aplicación: gestión de eventos, registro de participantes y control de aforo. Esta división facilita la localización de los casos, la asignación de responsables y la interpretación de los resultados. Además, permite agrupar casos que comparten precondiciones y objetos de prueba.
- ***Doce casos de prueba:*** el enunciado pide entre 10 y 12 casos. Se optó por 12 para cubrir las funcionalidades principales con al menos cuatro casos por suite. Esta cifra permite incluir casos positivos (creación con datos válidos), negativos (creación sin datos obligatorios, aforo igual a cero) y de borde (inscripción con una única plaza disponible). La distribución en 4+4+4 mantiene un equilibrio entre las tres áreas funcionales.
- ***Un único plan de pruebas:*** se creó un solo plan (Plan de Pruebas: Gestión de Eventos Universitarios) porque el proyecto se ejecuta en una única iteración. El plan asocia los 12 casos y define el contexto de ejecución.
- ***Un único build:*** se utilizó el build ***GEU v1.0*** para agrupar las ejecuciones realizadas. Al tratarse de una versión inicial, un solo build es suficiente para registrar los resultados. Si en el futuro se realizaran cambios en la aplicación, se podrían crear nuevos builds dentro del mismo plan.
- ***Ejecución con distintos resultados:*** se ejecutaron los casos y se registraron estados Passed, Failed y Blocked. Esto cumple el requisito del enunciado de obtener al menos un caso superado, uno fallido y uno bloqueado o no ejecutado. Además, permite observar cómo TESTLink almacena el historial de ejecuciones y cómo un mismo caso puede tener múltiples ejecuciones asociadas a distintos builds.

En conjunto, la estructura definida es mínima pero suficiente para analizar las entidades de TESTLink, sus atributos y sus relaciones, tal como se desarrollará en los apartados 4 y 5.

4.INVENTARIO DE ENTIDADES (PARA CADA ENTIDAD)

A continuación, mostraremos y analizaremos cada entidad individualmente:

4.1.Entidad 1

➔ ***Nombre:*** Proyecto de pruebas

➔ ***Descripción:*** Es la entidad principal que permite organizar las actividades de validación y pruebas de una aplicación. Actúa como contenedor de los elementos necesarios para gestionar las pruebas, estableciendo un contexto común para las suites, los casos y los planes de prueba.

En nuestro trabajo se ha creado el proyecto «Gestión de eventos universitarios», que agrupa las pruebas relacionadas con la creación de eventos, el registro de participantes y el control de aforos.

➔ ***Atributos principales:***

- ***Nombre:*** identifica el proyecto de pruebas.
- ***Prefijo:*** abreviatura utilizada para identificar los casos de prueba. En nuestro proyecto se utiliza GEU.
- ***Descripción o notas:*** recoge la finalidad del proyecto.
- ***Activo:*** indica si el proyecto está habilitado.
- ***Público:*** determina su visibilidad dentro de la herramienta.

➔ ***Relaciones observadas:***

- Un proyecto contiene suites de pruebas. En nuestro caso se han creado tres suites.
- Un proyecto permite crear planes de pruebas. Hemos definido un plan asociado a nuestro proyecto.
- Los casos de prueba se encuentran organizados dentro del proyecto mediante las suites.

4.2.Entidad 2

➔ ***Nombre:*** Suite de pruebas

➔ ***Descripción:*** Es una entidad utilizada para agrupar casos de prueba relacionados funcionalmente. Su finalidad es facilitar la organización y localización de las pruebas, evitando que todos los casos aparezcan en una única lista sin clasificación.

En nuestro proyecto se han creado tres suites: «Gestión de Eventos», «Registro de Participantes» y «Control de Aforos».

➔ ***Atributos principales:***

- ***Nombre:*** identifica la suite.
- ***Detalles o descripción:*** explica su finalidad y las funcionalidades que agrupa.
- ***Palabras clave:*** campo opcional que permite clasificar las pruebas.

➔ ***Relaciones observadas:***

- Cada una de las tres suites creadas pertenece al proyecto «Gestión de eventos universitarios».
- Una suite puede contener varios casos de prueba. En nuestra práctica, cada suite contiene cuatro casos.
- Las suites organizan los casos que posteriormente pueden incorporarse a un plan de pruebas.

4.3.Entidad 3

➔ ***Nombre:*** Caso de prueba

➔ ***Descripción:*** Representa una comprobación concreta que se desea realizar sobre una funcionalidad del sistema. Permite documentar qué se pretende verificar, cuáles son las condiciones necesarias para realizar la prueba y qué resultado se espera obtener.

Es uno de los elementos fundamentales de TestLink, ya que permite definir las comprobaciones que posteriormente serán planificadas y ejecutadas.

En nuestro proyecto se han creado doce casos de prueba, identificados desde ***GEU-1 hasta GEU-12***.

➔ ***Atributos principales:***

- ***Identificador:*** código que distingue cada caso de prueba, como GEU-1.
- ***Nombre o título:*** descripción breve de la funcionalidad que se comprueba.
- ***Resumen:*** explicación del objetivo del caso.
- ***Precondiciones:*** condiciones que deben cumplirse antes de realizar la prueba.
- ***Estado:*** situación del caso dentro de su proceso de definición, como Borrador o Final.
- ***Importancia:*** nivel de relevancia de la prueba, como Media o Alta.
- ***Tipo de ejecución:*** indica si la prueba es manual o automatizada.
- ***Duración estimada:*** tiempo previsto para realizarla.

➔ ***Relaciones observadas:***

- Cada caso creado se encuentra dentro de una suite de pruebas.
- Un caso de prueba puede tener versiones. Los casos creados muestran la versión 1.
- Los casos contienen los pasos necesarios para realizar sus comprobaciones, asociados a la versión correspondiente.
- Los casos pueden incorporarse a planes de pruebas. Hemos asociado los doce casos al plan creado.
- Un caso puede registrar varias ejecuciones. Esto se ha comprobado en el historial de GEU-1.

4.4.Entidad 4

➔ ***Nombre:*** Versión del caso de prueba

➔ ***Descripción:*** Representa una revisión concreta de la definición de un caso de prueba. Su finalidad es permitir identificar la versión de la especificación utilizada al planificar y ejecutar una prueba.

Durante la práctica se ha observado que TestLink identifica los casos mediante un número de versión. Por ejemplo, GEU-1 aparece como «Versión 1».

➔ ***Atributos principales:***

- ***Número de versión:*** identifica la revisión del caso.
- ***Caso de prueba asociado:*** indica a qué caso pertenece la versión.

➔ ***Relaciones observadas:***

- Cada versión corresponde a un caso de prueba determinado.
- Una versión contiene los pasos que definen cómo debe realizarse la prueba.
- Durante la asociación de casos a un plan, TestLink permite seleccionar la versión del caso que se desea incorporar.
- El historial de ejecuciones muestra la versión del caso utilizada.

4.5.Entidad 5

- ***Nombre:*** Paso de prueba
- ***Descripción:*** Representa una acción concreta que debe realizarse durante una prueba, junto con el resultado esperado de dicha acción.

  Los pasos permiten dividir un caso de prueba en operaciones ordenadas y verificables, facilitando su ejecución y la identificación de posibles errores.

  En nuestro proyecto se han documentado cuatro pasos para cada uno de los doce casos de prueba.
- ***Atributos principales:***
  - ***Número de paso:*** determina el orden de ejecución.
  - ***Acción:*** operación que debe realizar el usuario.
  - ***Resultado esperado:*** comportamiento que debería producir el sistema.
  - ***Tipo de ejecución:*** indica el modo de ejecución del paso.
- ***Relaciones observadas:***
  - Los pasos pertenecen a la definición de una versión de un caso de prueba.
  - Una versión de caso puede contener varios pasos. Por ejemplo, GEU-12 contiene cuatro pasos.
  - Durante la ejecución es posible registrar resultados y observaciones para los pasos individuales.

4.6.Entidad 6

➔ ***Nombre:*** Plan de pruebas

➔ ***Descripción:*** Es la entidad encargada de organizar y planificar la ejecución de un conjunto de casos de prueba. Permite seleccionar las comprobaciones que se realizarán y gestionar su ejecución sobre una versión determinada del software.

En nuestro trabajo se ha creado el plan «Plan de Pruebas - Gestión de Eventos Universitarios», al que se han incorporado los doce casos definidos.

➔ ***Atributos principales:***

- ***Nombre:*** identifica el plan de pruebas.
- ***Descripción:*** especifica los objetivos y el alcance del plan.
- ***Activo:*** indica si el plan se encuentra habilitado.
- ***Público:*** indica su visibilidad.

➔ ***Relaciones observadas:***

- El plan pertenece al contexto de un proyecto de pruebas.
- Un plan puede contener varios casos de prueba. Nuestro plan contiene doce.
- Un plan puede asociarse con builds. Hemos creado la build ***GEU V1.0.***
- Las ejecuciones se realizan dentro de un plan de pruebas seleccionado.

4.7.Entidad 7

➔ ***Nombre:*** Build

➔ ***Descripción:*** Representa una versión del software sobre la que se planifican y registran ejecuciones de prueba.

Su finalidad es identificar el contexto de versión en el que se obtienen los resultados, permitiendo diferenciarlos de los que puedan registrarse para otras versiones del producto.

Durante la práctica se ha creado la build «***GEU V1.0***», utilizada como versión de referencia para las ejecuciones demostrativas.

➔ ***Atributos principales:***

- ***Título:*** nombre identificativo de la build.
- ***Descripción:*** información sobre su finalidad.
- ***Activo:*** determina si está disponible para las operaciones de TestLink.
- ***Abierta:*** permite controlar si los resultados de prueba pueden modificarse.
- ***Fecha de lanzamiento:*** campo opcional para indicar la fecha de la versión.
- ***Datos adicionales de versión:*** campos opcionales como commit ID, tag, branch y release candidate.

➔ ***Relaciones observadas:***

- La build se crea dentro de un plan de pruebas. En nuestro caso está asociada al plan de Gestión de Eventos Universitarios.
- Una build puede tener varias ejecuciones de casos de prueba.
- Cada ejecución registrada se encuentra asociada a una build concreta. Los resultados demostrativos de nuestra práctica se han registrado bajo ***GEU V1.0.***

4.8.Entidad 8

➔ ***Nombre:*** Ejecución de prueba

➔ ***Descripción:*** Representa el registro de una ejecución de un caso de prueba dentro de un plan y una build determinados.

Su finalidad es almacenar los resultados obtenidos o registrados, permitiendo conocer el estado de las comprobaciones y consultar su historial.

Durante la práctica se han registrado ejecuciones demostrativas con los estados Pasado, Fallado y Bloqueado. Estos registros se utilizaron para observar el comportamiento de TestLink, sin ejecutar una aplicación real de gestión de eventos.

➔ ***Atributos principales:***

- ***Fecha y hora:*** momento en el que se registra la ejecución.
- ***Resultado:*** estado registrado, como Pasado, Fallado o Bloqueado.
- ***Usuario ejecutor:*** usuario que figura como responsable del registro.
- ***Versión del caso:*** revisión utilizada durante la ejecución.
- ***Modo de ejecución:*** manual en los casos de nuestra práctica.
- ***Notas o comentarios:*** información adicional sobre el resultado.
- ***Duración de ejecución:*** campo destinado a registrar el tiempo empleado.

➔ ***Relaciones observadas:***

- Una ejecución corresponde a un caso de prueba y a una versión concreta de este.
- La ejecución se registra dentro de un plan de pruebas y una build.
- Una ejecución está vinculada al usuario que figura en su historial.
- Un mismo caso puede tener varias ejecuciones. Se ha comprobado con GEU-1, cuyo historial conserva registros con diferentes resultados.
- Durante una ejecución pueden registrarse resultados individuales para los pasos del caso.

4.9.Entidad 9

➔ ***Nombre:*** Usuario

➔ ***Descripción:*** Representa a una persona o cuenta que utiliza TestLink para gestionar los elementos de un proyecto de pruebas.

Los usuarios permiten identificar quién participa en la definición, administración o ejecución de las pruebas y facilitan la asignación de responsabilidades.

Durante la práctica se ha trabajado desde una cuenta con rol de administrador y en el historial de ejecuciones aparece identificado el usuario que ha registrado los resultados.

➔ ***Atributos principales:***

- ***Nombre de usuario:*** identificador de la cuenta.
- ***Rol:*** conjunto de permisos asignados al usuario. En nuestra sesión se muestra el rol administrador.

➔ ***Relaciones observadas:***

- Los usuarios pueden figurar como responsables de los registros de ejecución.
- TestLink permite gestionar la asignación de casos a usuarios para su ejecución.
- El historial permite consultar qué usuario registró una ejecución determinada.

4.10.Entidad 10

➔ ***Nombre:*** Resultado de ejecución de paso

➔ ***Descripción:*** Representa la información registrada durante la comprobación individual de un paso perteneciente a un caso de prueba.

Esta entidad se propone a nivel conceptual para distinguir el resultado global del caso de los resultados específicos de sus pasos. No se ha comprobado si TestLink la almacena internamente como una entidad independiente.

➔ ***Atributos principales:***

- ***Estado del paso:*** resultado seleccionado durante la ejecución.
- ***Notas de ejecución:*** observaciones relativas al paso.
- ***Archivos adjuntos:*** evidencias que pueden incorporarse al registro, cuando corresponde.

➔ ***Relaciones observadas:***

- El resultado se registra para un paso determinado.
- Los estados y notas de los pasos se introducen dentro del contexto de ejecución de un caso.
- Una ejecución puede incluir información de resultado para varios pasos, tal como se observa en los formularios de ejecución utilizados.

5.DIAGRAMA ER

5.1.Diagrama completo

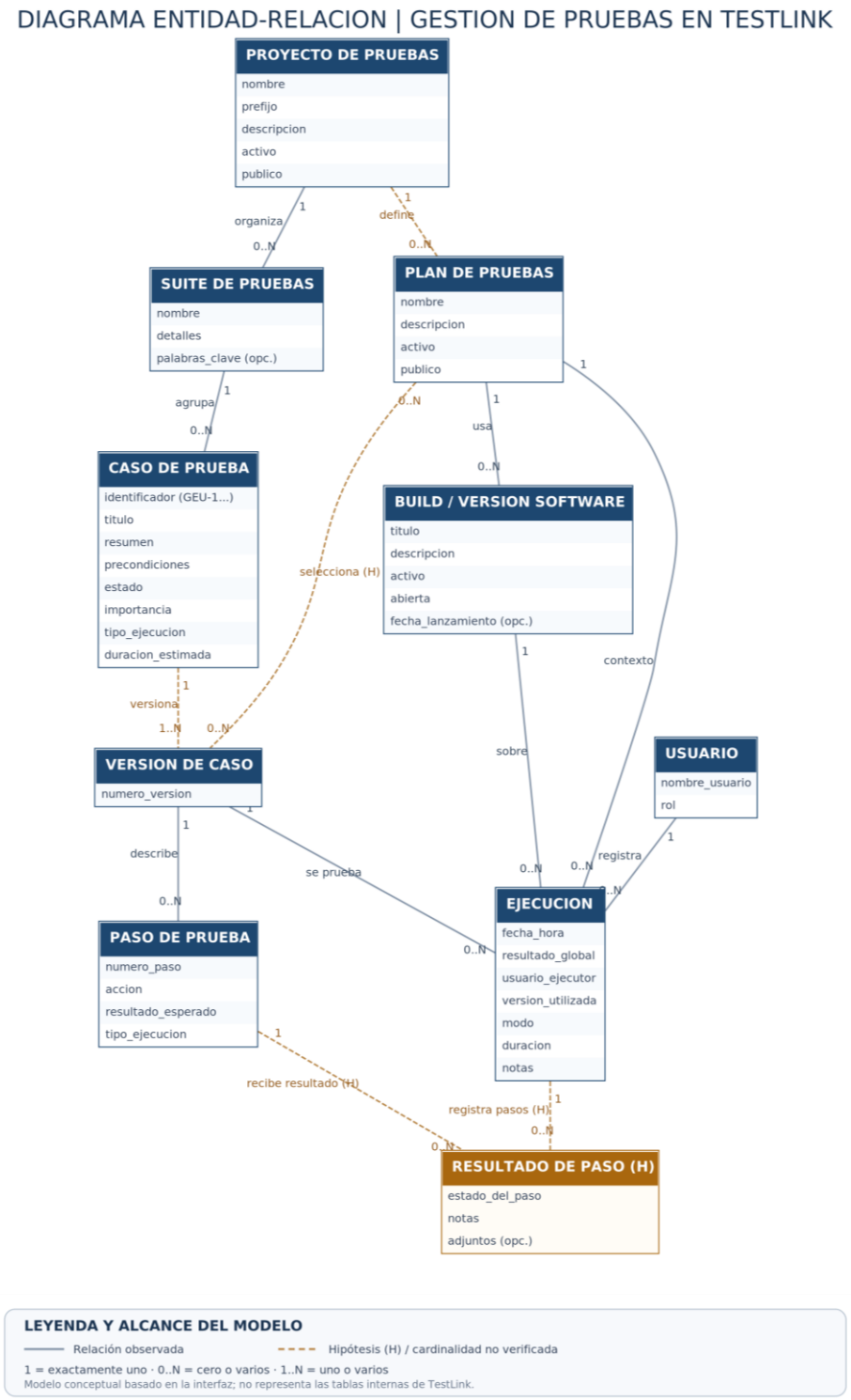


Ilustración 13 - Diagrama ER

5.2.Cardinalidades

Justificación de las cardinalidades:

Relación	Cardinalidad propuesta	Evidencia
Proyecto – Suite	1:N	Un proyecto contiene 3 suites
Suite – Caso	1:N	Cada suite contiene 4 casos
Caso – Versión	1:N	Se observó la versión 1; múltiples versiones son hipótesis
Versión – Paso	1:N	Cada versión documentada contiene 4 pasos
Proyecto – Plan	1:N	Se observó un plan; máximo N no comprobado
Plan – Build	1:N	Se creó GEU V1.0; máximo N no comprobado
Plan – Versión de caso	N:M (hipótesis)	Se asociaron 12 versiones al plan, pero no se probaron varios planes

Estas relaciones no han podido ser verificadas experimentalmente en su totalidad, por lo que se establecen como hipótesis del modelo, tal como se detalla en el apartado 5.3.

5.3.Hipótesis realizadas

Durante la elaboración del diagrama Entidad-Relación se han establecido diversas hipótesis para representar las relaciones y cardinalidades entre las entidades identificadas en TestLink.

Estas hipótesis son necesarias porque el modelo se ha construido a partir de la observación y utilización de la herramienta, sin consultar su base de datos ni su estructura interna. Por tanto, algunas relaciones se han podido comprobar directamente, mientras que otras se han deducido a partir del funcionamiento observado.

5.3.1.Pertenencia de suites y casos de prueba

Se ha considerado que cada suite de pruebas pertenece a un único proyecto y que cada caso de prueba se encuentra asociado a una suite dentro de dicho proyecto.

Esta decisión se basa en la estructura jerárquica observada en TestLink, donde el proyecto «Gestión de eventos universitarios» contiene tres suites y cada una de ellas agrupa cuatro casos de prueba.

Aunque esta organización se ha comprobado en nuestra práctica, no se han analizado otras configuraciones posibles, como la utilización de suites anidadas.

5.3.2.Versionado de los casos de prueba

Se ha supuesto que un mismo caso de prueba puede tener varias versiones, de modo que las modificaciones de su definición puedan conservarse sin perder las versiones anteriores.

Durante la práctica todos los casos creados muestran la versión 1. Por tanto, la posibilidad de disponer de múltiples versiones se ha considerado en el modelo, aunque no se ha comprobado mediante la creación de una segunda versión.

5.3.3.Relación entre planes y casos de prueba

Se ha planteado que un mismo caso de prueba, mediante una versión determinada, puede asociarse a varios planes de pruebas, dando lugar a una relación de muchos a muchos (N).

Durante la práctica se comprobó que el plan creado podía contener varios casos de prueba, concretamente doce. Sin embargo, únicamente se creó un plan, por lo que no se ha verificado experimentalmente que un mismo caso pueda estar asociado simultáneamente a varios planes.

Por este motivo, la cardinalidad N se considera una hipótesis del modelo.

5.3.4.Relación entre proyectos, planes y builds

Se ha supuesto que un proyecto puede contener varios planes de pruebas y que cada plan puede disponer de varias builds.

En nuestra práctica únicamente se creó un plan y una build denominada «***GEU V1.0***». Por tanto, se ha comprobado la existencia de estas asociaciones, pero no su cardinalidad máxima.

5.3.5.Registro de múltiples ejecuciones

Se ha considerado que una versión de un caso de prueba puede tener diferentes registros de ejecución, conservando su historial.

Esta relación sí se ha comprobado directamente, puesto que TestLink permitió registrar varios resultados para GEU-1 y consultar posteriormente las ejecuciones anteriores.

También se ha considerado que cada ejecución queda vinculada al contexto de un plan, una build y una versión del caso de prueba.

5.3.6.Resultados individuales de los pasos

Se ha propuesto representar el resultado de ejecución de un paso como una entidad conceptual diferenciada de la ejecución global.

Esta decisión se debe a que TestLink permite seleccionar un estado y escribir observaciones para los pasos individuales de un caso de prueba.

Sin embargo, no se ha comprobado si dichos resultados se almacenan internamente como entidades independientes. Por tanto, su representación constituye una hipótesis de diseño conceptual.

5.3.7.Participación de los usuarios

Se ha considerado que un usuario puede registrar múltiples ejecuciones y que cada registro identifica al usuario que lo ha realizado.

Esta decisión se apoya en el historial de ejecuciones, donde aparece el usuario responsable del registro.

Asimismo, se ha diferenciado el usuario que registra una ejecución del usuario que puede estar asignado como responsable de realizarla, ya que TestLink muestra información específica sobre la asignación de testers.

5.3.8.Cardinalidades mínimas

Para determinadas relaciones se ha considerado la posibilidad de que una entidad exista antes de tener elementos asociados.

Por ejemplo, durante la construcción del proyecto se creó una suite antes de incorporar casos de prueba, y también se creó un plan antes de asociarle los doce casos.

Estas observaciones justifican que determinadas relaciones puedan tener una participación mínima de cero.

No obstante, no se han comprobado todas las restricciones mínimas y máximas de las entidades, por lo que las cardinalidades no verificadas deben entenderse como propuestas del modelo.

5.4.Relaciones observadas

El diagrama Entidad-Relación se ha elaborado a partir de la observación y utilización de TestLink durante la práctica, identificando las entidades necesarias para organizar, planificar y ejecutar pruebas de software.

La estructura del modelo se justifica mediante las siguientes observaciones:

- ***Proyecto, Suite y Caso de prueba:*** se ha creado un proyecto con tres suites y doce casos de prueba, demostrando su organización jerárquica.
- ***Versión del caso y Paso de prueba:*** los casos disponen de una versión identificada y contienen varios pasos con sus resultados esperados.
- ***Plan de pruebas y Build:*** se ha creado un plan con doce casos asociados y una build denominada ***GEU V1.0.***
- ***Ejecución y Usuario:*** el historial permite consultar diferentes ejecuciones de un mismo caso, indicando su fecha, resultado y usuario.
- ***Resultado de ejecución de paso:*** se ha incluido como entidad conceptual para representar los resultados individuales de los pasos.

Las cardinalidades se han establecido a partir de estas observaciones. Por ejemplo, una suite contiene varios casos y un caso puede registrar múltiples ejecuciones. Sin embargo, algunas relaciones, como la posible asociación de un caso a varios planes de pruebas, se han considerado hipótesis al no haberse comprobado directamente.

6.REFLEXIÓN TEMPORAL

6.1.Valoración crítica

La realización de esta práctica nos ha permitido comprender mejor el funcionamiento de TestLink como herramienta de gestión de pruebas de software. Hemos aprendido a organizar un proyecto mediante suites, casos de prueba, planes, versiones y ejecuciones.

Consideramos que el plan de pruebas es uno de los elementos más importantes, ya que permite organizar los casos y gestionar sus ejecuciones. Una de las principales dificultades ha sido identificar correctamente las relaciones y cardinalidades del modelo Entidad-Relación, especialmente la relación entre los planes y los casos de prueba.

Como aspecto positivo, TestLink permite centralizar la información, mantener un historial de resultados y organizar las pruebas de forma más eficiente que mediante documentos manuales. Sin embargo, su interfaz puede resultar poco intuitiva al principio.

6.2.Propuestas de mejora

Tras utilizar TestLink, consideramos que podrían introducirse las siguientes mejoras:

- ***Interfaz más intuitiva:*** simplificar los menús y facilitar la localización de las funcionalidades principales.
- ***Mejor organización visual:*** representar de forma más clara las relaciones entre proyectos, suites, casos y planes de prueba.
- ***Mayor facilidad de uso:*** simplificar la creación de casos y la introducción de pasos y resultados esperados.
- ***Documentación más accesible:*** incorporar explicaciones dentro de la herramienta para facilitar su aprendizaje.

Estas mejoras permitirían reducir el tiempo necesario para familiarizarse con TestLink y facilitarían su utilización por parte de usuarios sin experiencia previa.

6.3.Lecciones aprendidas

La principal lección aprendida ha sido comprender la importancia de planificar y documentar correctamente las pruebas antes de ejecutarlas.

También hemos aprendido a diferenciar los casos de prueba de sus ejecuciones, a utilizar builds y a interpretar las relaciones entre las distintas entidades de TestLink.

La elaboración del diagrama Entidad-Relación nos ha ayudado a comprender cómo se organiza la información dentro de la herramienta. Para justificar las cardinalidades hemos utilizado evidencias como la estructura de las suites, la asociación de doce casos al plan y el historial de ejecuciones.

Por último, hemos comprendido la importancia de distinguir entre resultados reales y simulados, ya que durante la práctica únicamente disponíamos de TestLink y no de una aplicación funcional sobre la que realizar las pruebas.